

CS462 – Big Data Systems

Lab 3: Apache Hadoop & MapReduce

Bellevue College – Spring 2026

Course	CS462 – Big Data Systems
Lab Number	Lab 3
Topics	Apache Hadoop, YARN, MapReduce, Java MapReduce API, Combiners
Estimated Time	3–4 hours
Points	100
Prerequisite	Lab 2 – HDFS & Fault Tolerance (cluster setup is the same)
Submission	Canvas – upload a single ZIP file (see Deliverables section)

Learning Objectives

By the end of this lab you will be able to:

- Reuse the Docker Compose Hadoop cluster from Lab 2
- Upload and manage real-world input data (a corpus of public-domain books and a 1995 NASA web server log) in HDFS
- Run the built-in WordCount example and submit it to YARN
- Implement a Mapper and Reducer in Java against the Hadoop MapReduce API
- Compile and package your Java code into a JAR and submit it to YARN
- Apply MapReduce to a real web-server access log to count HTTP status codes
- Use a Combiner to reduce shuffle volume and observe its effect via Hadoop counters
- Read and interpret MapReduce job output stored in HDFS
- Monitor a running job and review its history using the YARN ResourceManager web UI

Prerequisites

- Docker Desktop installed and running (version 20.10 or later)
- Docker Compose v2 available
- At least 6 GB of free RAM allocated to Docker
- The hadoop-lab folder from Lab 2 (docker-compose.yml and hadoop.env)

Note: If you no longer have your Lab 2 files, refer to the *Environment Setup* section in the Lab 2 handout to recreate them before continuing.

Environment Setup

This lab reuses the Docker Compose cluster from Lab 2, but the `hadoop.env` from Lab 2 needs several additions and one important fix before MapReduce will work. The `apache/hadoop:3.4.3` image ships a minimal configuration: it does not export `HADOOP_HOME`, the default YARN application classpath does not include the MapReduce jars, and the env vars that bootstrap the MapReduce ApplicationMaster are not pre-set. Without the changes below, every MapReduce job fails at submission with `ClassNotFoundException`: `org.apache.hadoop.mapreduce.v2.app.MRAppMaster`.

Replace the contents of your `hadoop.env` with the following:

```
HADOOP_HOME=/opt/hadoop
CORE-SITE.XML_fs.defaultFS=hdfs://namenode
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=1
MAPRED-SITE.XML_mapreduce.framework.name=yarn
MAPRED-SITE.XML_mapreduce.application.classpath=/opt/hadoop/share/hadoop/
mapreduce/*:/opt/hadoop/share/hadoop/mapreduce/lib/*
MAPRED-SITE.XML_yarn.app.mapreduce.am.env=HADOOP_MAPRED_HOME=/opt/hadoop
MAPRED-SITE.XML_mapreduce.map.env=HADOOP_MAPRED_HOME=/opt/hadoop
MAPRED-SITE.XML_mapreduce.reduce.env=HADOOP_MAPRED_HOME=/opt/hadoop
YARN-SITE.XML_yarn.resourcemanager.hostname=resourcemanager
YARN-SITE.XML_yarn.nodemanager.aux-services=mapreduce_shuffle
YARN-SITE.XML_yarn.nodemanager.pmem-check-enabled=false
YARN-SITE.XML_yarn.nodemanager.vmem-check-enabled=false
YARN-SITE.XML_yarn.application.classpath=/opt/hadoop/etc/hadoop:/opt/hadoop/
share/hadoop/common/lib/*:/opt/hadoop/share/hadoop/common/*:/opt/hadoop/share/
hadoop/hdfs:/opt/hadoop/share/hadoop/hdfs/lib/*:/opt/hadoop/share/hadoop/hdfs/
*/opt/hadoop/share/hadoop/yarn/lib/*:/opt/hadoop/share/hadoop/yarn/*:/opt/
hadoop/share/hadoop/mapreduce/lib/*:/opt/hadoop/share/hadoop/mapreduce/*
```

Note: The `CORE-SITE.XML`, `HDFS-SITE.XML`, `MAPRED-SITE.XML`, and `YARN-SITE.XML` prefixes are the `apache/hadoop` convention: the image's startup helper turns each prefixed line into a `<property>` block in the matching `*-site.xml` file. The unprefixed `HADOOP_HOME=/opt/hadoop` line is passed through as a regular container env var and is inherited by every container that sources `hadoop.env`. Note that all paths use literal `/opt/hadoop` rather than `$HADOOP_HOME` — Hadoop's variable-substitution mechanism only expands a fixed allowlist (`HADOOP_COMMON_HOME`, `HADOOP_MAPRED_HOME`, `HADOOP_YARN_HOME`, `JAVA_HOME`, etc.), and `HADOOP_HOME` is not on it, so `$HADOOP_HOME` would survive Hadoop's substitution as a literal string and break the classpath. The `yarn.application.classpath` line is the one that actually matters for fixing the `MRAppMaster ClassNotFoundException`: it adds the `share/hadoop/mapreduce/*` jars to the classpath that bootstraps every YARN container, including the AM.

In Parts 2–4 you will edit Java files and compile them inside the namenode container. Rather than copying files back and forth with `docker cp`, mount a host directory directly so you can edit with your IDE on the laptop and see the changes immediately at `/workspace` inside the container. Add the following volumes block to the namenode service in `docker-compose.yml` (next to the existing image, command, ports, and `env_file` entries):

```
volumes:
  - ./workspace:/workspace
```

Then create the workspace folder next to `docker-compose.yml` so the bind mount has something to point at:

```
mkdir -p workspace
```

Note: The bind mount is bidirectional. Files you edit in your IDE on the laptop appear instantly inside the container, and the .class files and JAR archives that `javac` and `jar` create inside the container appear in `workspace/` on the laptop — convenient for inspecting and zipping up your final submission.

With `hadoop.env` updated and the workspace mount in place, start the cluster:

```
docker compose up -d
```

Note: If your cluster from Lab 2 was already running before you edited `hadoop.env`, run `docker compose down` first; `docker compose up -d` alone will not pick up env-file changes for already-running containers.

Note: Apple Silicon Mac (M1/M2/M3): If the containers fail to start with a platform mismatch error, ensure each service in your `docker-compose.yml` has the line `"platform: linux/amd64"` directly below its `image:` line. See the Lab 2 handout for the corrected file.

Wait about 30 seconds, then confirm the cluster is healthy:

- NameNode Web UI – `http://localhost:9870`
- YARN ResourceManager UI – `http://localhost:8088`

Open a shell on the NameNode – you will use this for all HDFS and Hadoop commands throughout the lab:

```
docker exec -it hadoop-lab-namenode-1 bash
```

Because `hadoop.env` now includes `HADOOP_HOME=/opt/hadoop`, every container — including the shell you just opened — inherits that variable. Confirm it before going further:

```
echo "HADOOP_HOME=$HADOOP_HOME"
```

You should see `HADOOP_HOME=/opt/hadoop`. If it is empty, `hadoop.env` was not picked up — verify the file is in the same folder as `docker-compose.yml`, and that you ran `docker compose down` before `docker compose up -d` so the new env file took effect.

Part 1: WordCount on a Real Corpus (15 points)

In this warm-up part you will run the WordCount example that ships with Hadoop. Instead of a tiny sample file, you will use a corpus of full-length public-domain books from Project Gutenberg. The same input data is reused later in Part 2, so make sure this part finishes cleanly before moving on.

1.1 Prepare the Project Gutenberg Corpus

Create an input directory in HDFS and download five public-domain books into a temporary folder inside the namenode container. The Project Gutenberg URL pattern below is stable and works without authentication:

```
hdfs dfs -mkdir -p /user/student/input/gutenberg

mkdir -p /tmp/gutenberg && cd /tmp/gutenberg
for ID in 100 2701 2600 1342 1661; do
  wget -q -O pg${ID}.txt https://www.gutenberg.org/cache/epub/${ID}/pg${ID}.txt
done
ls -lh /tmp/gutenberg
```

That downloads roughly 12 MB of plain text: the Complete Works of Shakespeare (100), Moby-Dick (2701), War and Peace (2600), Pride and Prejudice (1342), and the Adventures of Sherlock Holmes (1661). Verify the file sizes look reasonable, then upload all five into HDFS:

```
hdfs dfs -put -f /tmp/gutenberg/*.txt /user/student/input/gutenberg/
hdfs dfs -ls -h /user/student/input/gutenberg
```

Note: If a Project Gutenberg URL ever returns an empty file (rate limiting), wait a minute and retry. *wget* exits 0 even on a 0-byte response, so always check sizes with *hdfs dfs -ls -h*.

1.2 Run the Built-in WordCount Job

Submit the example WordCount job to YARN. It will read every file under `/user/student/input/gutenberg` and write sorted word-frequency pairs to the output directory:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \
  wordcount /user/student/input/gutenberg /user/student/output/wordcount-
builtin
```

A successful run ends with a line similar to:

```
Job job_XXXX_0001 completed successfully
```

Take a screenshot of the terminal showing the job completion message. Label it `screenshot-1a.png`.

1.3 Inspect the Results

List the output directory and print the top-10 most frequent tokens:

```
hdfs dfs -ls /user/student/output/wordcount-builtin
```

```
hdfs dfs -cat /user/student/output/wordcount-builtin/part-r-00000 | sort -k2  
-rn | head -10
```

Record the top-10 list and take a screenshot of the terminal output. Label it screenshot-1b.png.

***Note:** The default WordCount tokenizer is case-sensitive and splits only on whitespace, so The and the count separately and punctuation stays attached to words. You will fix that in Part 2 by writing your own mapper.*

Part 1 — Deliverables

- screenshot-1a.png — terminal showing job completion message
- screenshot-1b.png — top-10 word frequencies in the terminal
- In your write-up: list the top-10 words and their counts, and one sentence on what surprises you about that list (it will look noisy)

Part 2: Write Your Own WordCount in Java (30 points)

Running someone else's JAR is a useful starting point, but the heart of MapReduce is writing your own Mapper and Reducer. In this part you will fill in the two TODO blocks in WordCount.java, compile the program inside the namenode container, package it as a JAR, and submit it to YARN.

The template fixes a real problem with the built-in tokenizer: yours will lower-case every line and split on any non-letter character so The, the, and the. all collapse to the same key.

2.1 Get the Template into the Container

On your host laptop, copy WordCount.java from the lab3_templates folder I provided into the workspace folder you mounted earlier:

```
cp lab3_templates/WordCount.java workspace/
```

Then, back in your namenode shell, confirm the file is visible inside the container at /workspace/:

```
cd /workspace
ls -l WordCount.java
```

Note: From now on, edit WordCount.java directly in workspace/ on your host using your IDE of choice (VS Code, IntelliJ IDEA, vim — anything). Saving the file in your IDE makes the change visible at /workspace/WordCount.java inside the namenode container instantly. If you closed your namenode shell, reopen it with: `docker exec -it hadoop-lab-namenode-1 bash`. Always run the build steps from /workspace, not /root.

2.2 Fill in the Two TODO Blocks

Open workspace/WordCount.java on your host in your IDE of choice and complete the two TODO blocks. The file already includes the imports, the main() method that wires up the job, and the Mapper/Reducer class skeletons; you only need to write the body of map() and reduce(). The hints inside each TODO block describe the expected behavior.

Briefly: TODO 1 should tokenize the input line and call context.write(word, ONE) once per token. TODO 2 should sum the per-key values and emit (key, total).

Note: Reuse the field-level Text and IntWritable instances rather than allocating new ones inside the loop. Hadoop calls map() once per input record (potentially millions of times), and per-call object allocation is one of the most common performance pitfalls in introductory MapReduce code.

2.3 Compile and Package

Hadoop ships its full Java classpath through the hadoop classpath helper. Compile WordCount.java against that classpath, then jar up the resulting class files:

```
cd /workspace
mkdir -p build
javac -cp "$(hadoop classpath)" -d build WordCount.java
```

```
jar cf wordcount.jar -C build .  
ls -l wordcount.jar
```

You should see a wordcount.jar of a few kilobytes. Inspect what is inside if you are curious:

```
jar tf wordcount.jar
```

Take a screenshot of the successful javac + jar commands and the resulting wordcount.jar listing. Label it screenshot-2a.png.

2.4 Submit Your JAR to YARN

Run your custom job against the same Project Gutenberg corpus you uploaded in Part 1:

```
hadoop jar wordcount.jar WordCount \  
  /user/student/input/gutenberg \  
  /user/student/output/wordcount-custom
```

Note: If you re-run the job after a fix, delete the old output directory first: `hdfs dfs -rm -r -f /user/student/output/wordcount-custom`. Hadoop refuses to overwrite an existing output directory by design — that is a feature, not a bug.

2.5 Compare Your Output Against the Built-in Job

Print the top-10 words from your custom job and compare them with the Part 1 output:

```
hdfs dfs -cat /user/student/output/wordcount-custom/part-r-000000 | sort -k2 -rn  
| head -10
```

Take a screenshot of the top-10 list and label it screenshot-2b.png.

Part 2 — Deliverables

- Your completed WordCount.java (include in the ZIP)
- screenshot-2a.png — successful compile and JAR creation
- screenshot-2b.png — top-10 list from your custom job
- In your write-up: a 2–3 sentence comparison of your top-10 list vs. the Part 1 list. Why are the counts different even though the input is identical?

Part 3: Real Web Logs — NASA HTTP Status Codes (30 points)

WordCount-style aggregation is a useful first lesson, but real MapReduce jobs are usually applied to logs and structured data. In this part you will run a custom MapReduce job over the NASA-HTTP dataset: every HTTP request handled by the public NASA Kennedy Space Center web server during July and August 1995. The dataset is a real-world Common Log Format access log of about 3.5 million requests.

3.1 Download and Stage the NASA Access Log

Inside the namenode container, fetch the July 1995 NASA log from the Lawrence Berkeley Lab Internet Traffic Archive and stage it into HDFS:

```
cd /tmp
wget -q https://ita.ee.lbl.gov/traces/NASA_access_log_Jul95.gz
ls -lh NASA_access_log_Jul95.gz
gunzip -k NASA_access_log_Jul95.gz
wc -l NASA_access_log_Jul95

hdfs dfs -mkdir -p /user/student/input/nasa
hdfs dfs -put -f NASA_access_log_Jul95 /user/student/input/nasa/
hdfs dfs -ls -h /user/student/input/nasa
```

You should end up with a single file of about 200 MB and roughly 1.9 million lines in HDFS. Inspect a few records so you understand the format your mapper will be parsing:

```
hdfs dfs -cat /user/student/input/nasa/NASA_access_log_Jul95 | head -3
```

Each line follows the Common Log Format:

```
host - - [timestamp] "METHOD path PROTOCOL" status bytes
```

***Note:** If your laptop is short on disk space, you can skip the .gz copy with `rm NASA_access_log_Jul95.gz` once the uncompressed file is in HDFS.*

3.2 Fill in StatusCodeCount.java

Copy the second template into your workspace folder so it appears at `/workspace/StatusCodeCount.java` inside the namenode container, then fill in the two TODO blocks in your IDE. The mapper has to parse a log line, extract the HTTP status code, and emit (statusCode, 1). The reducer is structurally identical to the WordCount reducer from Part 2 — copy your solution from there.

```
# on your host laptop:
cp lab3_templates/StatusCodeCount.java workspace/
```

Open the file in the container, fill in TODO 1 and TODO 2, then compile and package it the same way as Part 2:

```
cd /workspace
mkdir -p build_status
javac -cp "$(hadoop classpath)" -d build_status StatusCodeCount.java
jar cf statuscount.jar -C build_status .
```

Note: Real-world data is messy. The provided regex skips lines that do not match the expected format. Use the CS462/MalformedLines counter (already shown in the TODO hint) to count how many lines were dropped — you will report that number in your write-up.

3.3 Run the Job and Inspect the Output

```
hadoop jar statuscount.jar StatusCodeCount \  
  /user/student/input/nasa \  
  /user/student/output/nasa-status 2>&1 | tee statuscount.log
```

```
hdfs dfs -cat /user/student/output/nasa-status/part-r-000000
```

You should see a small table of HTTP status codes (200, 304, 302, 404, 403, 500, ...) and the number of times each occurred. Take a screenshot of the printed output and label it screenshot-3a.png.

3.4 Read the Counters from the Job Output

When the StatusCodeCount job completed, Hadoop printed a Counters: NN block at the bottom of the terminal. The pipe through tee in the previous step also captured all of that output to /workspace/statuscount.log, which survives across shell sessions and is visible on your laptop in the same workspace folder.

Print just the Counters section from the log:

```
grep -A60 "Counters:" statuscount.log
```

The output is grouped by counter family (File System Counters, Map-Reduce Framework, Job Counters, ...) plus your custom CS462 group. Note these specific values: CS462 / MalformedLines (your custom counter, how many log lines failed to match the regex), Map input records, and Reduce output records (both under Map-Reduce Framework).

Take a screenshot of the terminal showing the Counters block, with at least the Map-Reduce Framework and CS462 sections visible. Label it screenshot-3b.png.

Note: In a production cluster you would normally view counters in the JobHistoryServer web UI. The cluster you reused from Lab 2 doesn't include that service, so the terminal log is your authoritative source. Counters are also a job's standard observability artifact — even with the UI, every Hadoop release prints them at the end of every job for exactly this reason.

Part 3 — Deliverables

- Your completed StatusCodeCount.java (include in the ZIP)
- screenshot-3a.png — the printed status-code histogram from HDFS
- screenshot-3b.png — terminal output of `grep -A60 "Counters:" statuscount.log`, showing the CS462 group and the Map-Reduce Framework section
- In your write-up: the full list of (status code, count) pairs your job produced, plus a one-sentence interpretation of the most common non-200 status code

Part 4: Optimizing the Shuffle with a Combiner (20 points)

A Combiner is a mini-reducer that runs on the map side, on each map output partition, before the data is shuffled across the network. For commutative-and-associative aggregations like sum, plugging the Reducer in as the Combiner can dramatically reduce the bytes moved across the cluster. In this part you will measure that effect.

4.1 Capture Baseline Counters (no Combiner)

Hadoop printed counters at the end of your Part 2 wordcount-custom job, but unless you redirected the output we did not save them. Re-run the wordcount-custom job now with the output piped through tee, so you have a permanent baseline log to compare against:

```
hdfs dfs -rm -r -f /user/student/output/wordcount-custom
hadoop jar wordcount.jar WordCount \
  /user/student/input/gutenberg \
  /user/student/output/wordcount-custom 2>&1 | tee baseline.log
```

Once it finishes, find the Counters: NN block in baseline.log (use `grep -A60 "Counters:" baseline.log` if you want to print just that section). Write down the following four numbers:

- Map output records
- Map output bytes
- Reduce shuffle bytes (the bytes pulled from mappers to reducers)
- Combine input records (will be 0 here — no combiner ran)

4.2 Enable the Combiner

Reopen WordCount.java in the namenode container and uncomment the single line in main() that registers the combiner:

```
// before:
// job.setCombinerClass(IntSumReducer.class);

// after:
job.setCombinerClass(IntSumReducer.class);
```

This works because your IntSumReducer just sums IntWritable values, which is associative and commutative. Hadoop is allowed to call the combiner zero, one, or many times per partition; the final sum is the same either way.

Rebuild and re-run, writing to a new output directory so you can compare the two outputs:

```
cd /workspace
rm -rf build && mkdir -p build
javac -cp "$(hadoop classpath)" -d build WordCount.java
jar cf wordcount-combiner.jar -C build .

hadoop jar wordcount-combiner.jar WordCount \
  /user/student/input/gutenberg \
  /user/student/output/wordcount-combiner 2>&1 | tee combiner.log
```

4.3 Compare Counters

Open combiner.log and find the same four counters you recorded from baseline.log. Combine input records should now be non-zero (a combiner ran), and Reduce shuffle bytes should be noticeably smaller than the baseline. To see the two Counters blocks side by side:

```
diff <(grep -A60 "Counters:" baseline.log) <(grep -A60 "Counters:"  
combiner.log) | head -40
```

Confirm the actual word counts are unchanged — combining is a pure optimization, not a semantic change:

```
hdfs dfs -cat /user/student/output/wordcount-combiner/part-r-000000 | wc -l  
hdfs dfs -cat /user/student/output/wordcount-custom/part-r-000000 | wc -l  
  
# spot-check the top words match:  
hdfs dfs -cat /user/student/output/wordcount-combiner/part-r-000000 | sort -k2  
-rn | head -5
```

Take a screenshot of the diff output (or of the combiner.log Counters block on its own) and label it screenshot-4a.png. Take a separate screenshot of the spot-check terminal output showing the matching line counts and top-5 words and label it screenshot-4b.png.

Part 4 — Deliverables

- Your final WordCount.java with the combiner enabled (rename it WordCount_Combiner.java in the ZIP so we can see both versions if you saved a copy)
- screenshot-4a.png — terminal output of the diff between baseline.log and combiner.log Counters blocks (or of the combiner.log Counters section directly)
- screenshot-4b.png — terminal showing line counts and matching top-5 words
- In your write-up: a small table comparing the four counters (no combiner vs. combiner), and one sentence on why this optimization is safe for sum but would NOT be safe for, say, computing a median

Tearing Down the Cluster

When you are finished, shut down all containers and free the resources:

```
docker compose down
```

Note: This removes the containers but does not delete your docker-compose.yml or hadoop.env files. You can bring the cluster back up at any time by running docker compose up -d again.

Submission Instructions

Create a ZIP file named lab3_<YourLastName>_<YourFirstName>.zip containing:

1. A write-up document (PDF or DOCX) that includes:
 - Your name, student ID, and the date
 - A brief description (3–4 sentences) of what you did
 - The top-10 word frequencies from Part 1 and Part 2 (and the one-sentence comparison from Part 2)
 - The full status-code histogram from Part 3 plus the one-sentence interpretation
 - The four-counter comparison table from Part 4
 - Answers to the reflection questions below
2. All screenshots: screenshot-1a, 1b, 2a, 2b, 3a, 3b, 4a, 4b
3. Your completed WordCount.java and StatusCodeCount.java (and a copy of the combiner version if you saved one separately)
4. Your docker-compose.yml and hadoop.env files

Reflection Questions

Answer each question in your write-up (3–5 sentences each):

- What are the Map and Reduce phases in MapReduce, and what does the Shuffle in between actually do? Walk through what happens to one line of a Project Gutenberg book as it travels through your Part 2 WordCount job.
- Why does the WordCount Reducer also work as a Combiner? Give one concrete example of a different aggregation where reusing the reducer as a combiner would produce wrong results, and explain why.
- What is the role of YARN in the Hadoop ecosystem, and how does it differ from HDFS? Where in your Part 3 job did each one show up?
- Real-world datasets are dirty. How did your StatusCodeCount mapper handle malformed lines, and what would happen to your job if you let a NumberFormatException escape from map() instead?
- MapReduce writes its intermediate and final output to disk (and HDFS) rather than keeping it in memory. Name one advantage and one disadvantage of that design choice, particularly versus a system like Spark that prefers to keep data in memory.

Grading Rubric

Task / Deliverable	Points	Grading Criteria
Setup — cluster running & inputs in HDFS	5	Both /user/student/input/gutenberg and /user/student/input/nasa are populated; cluster shows healthy in the YARN UI
Part 1 — built-in WordCount on Gutenberg corpus	15	Job completes; top-10 word list reported in write-up; screenshots 1a and 1b included
Part 2 — custom WordCount.java compiles and runs	15	Submitted .java file compiles cleanly; both TODO blocks implemented correctly; YARN job succeeds; screenshots 2a and 2b included
Part 2 — comparison write-up	15	Top-10 list from custom job included; correct explanation of why counts differ from Part 1 (case-folding, punctuation handling)
Part 3 — StatusCodeCount.java compiles and runs	20	Both TODO blocks implemented; regex parsing works; full status-code histogram in write-up; screenshots 3a and 3b included
Part 3 — interpretation	10	Correct interpretation of the most common non-200 status code; MalformedLines counter value reported
Part 4 — Combiner enabled and measured	15	Counter table comparing no-combiner vs. combiner included; counts match between the two output directories; screenshots 4a and 4b included
Reflection questions	5	All five reflection questions answered with a few sentences each
TOTAL	100	

Note: The due date for this lab is posted on Canvas. Submissions received after that date will receive reduced points in accordance with the course's late submission policy.